

As you set forth on the Infinite Adventure project there are several stumbling points that can arise. The questions on this week's section are geared to help smooth over and alleviate those potential problems!

1. Compound and nested data types can be incredibly flexible in the data they can hold, but they can be complicated or confusing to work with. You often need to track multiple indexes as things become nested, and it is imperative to track what data types are representing different parts of the data structure. To that end, this problem is designed to help you get some practice in working with these types of structures.

In the starting repository is a file names `space_missions.json`. This file contain data on a host of fictional space missions. Each mission has a name, a crew, and a collection of hardware that was tested. Each crew member has a name, a role on that mission, and set list of certifications. In contrast, each piece of hardware has a unique identifier, and general category, and some diagnostics. Each diagnostic has a status associated with it and then an amount of power that was measured, in watts. Figure 1 below tries to give you a visual of the different nested pieces and their various types.

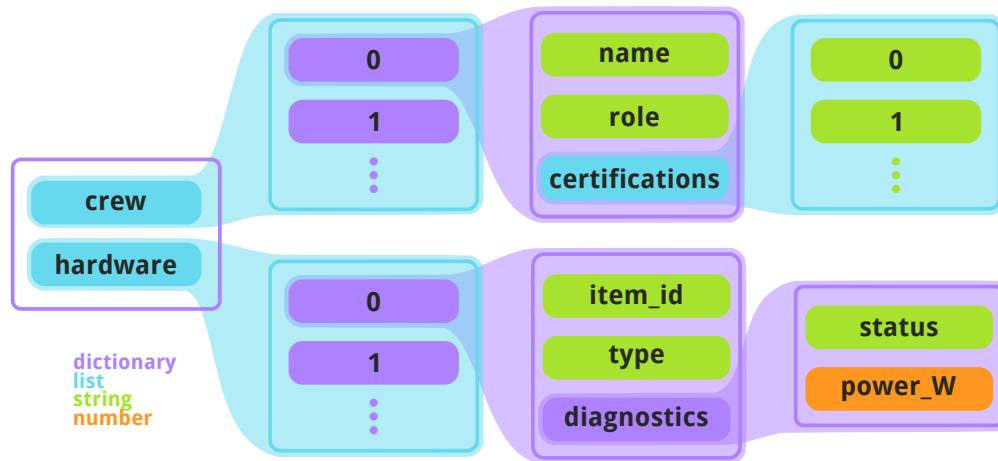


Figure 1: A color coordinated visual depiction of the internal structure of the space missions JSON data. Purple represents dictionaries, blue lists, green strings, and orange numbers.

Your challenge in this problem will be three-fold: to load in the data and then use it to answer two questions.

- (a) Write a function called `load_data` that loads in the data from the JSON file and returns the corresponding Python dictionary. This should be straightforward and you just need to open the file (in read mode) and then utilize the `json` library.
- (b) Here you need to write a function called `eva_count` which counts (and returns) the number of *different* crew members who have an “EVA” certification and who participated in a mission that began with an “A”. It is possible for the same crew member to serve on multiple missions, so make sure you don’t double count! Your function should take the dictionary generated by Part A as its input.

- (c) Write one last function called `power_missions` that takes the dictionary from Part A as its input. This function should return a *set* of all the mission names when the *Navigation* type of hardware tested during the mission averaged over 100 watts of power.
2. The Infinite Adventure project leverages having ChatGPT automatically generate rooms or scenes that a player tries to visit but which do not already exist. To do so, your Python code needs to “contact” and exchange information with a piece of software (ChatGPT) that is running on the web. The NotOpenAI library connects to one such API. So long as we interact with the API properly, it will give us the desired information, in the same way that if we call a function correctly it will return our desired information. To work with the NotOpenAI library, the general syntax looks like:

```
CLIENT = NotOpenAI(api_key="yourapikey") # Create the client

chat_completion = CLIENT.chat.completions.create(
    messages=[
        {
            'role': 'user',
            'content': Your prompt string
        }
    ],
    model="gpt-4o-mini",
    response_format={"type": "json_object"} # if json
)
response_str = chat_completion.choices[0].message.content
Convert response string to Python dict
```

You should have already been sent your NotOpenAI API key, which you will need to provide at the top.

Your goal in this problem is to use the NotOpenAI API to retrieve a JSON structure that contains information on the 5 most influential adventure games of all time. Your structure should include:

- The game name
- The game creator
- The year the game was first published
- and a one sentence description of why it was influential.

Because LLMs could return this information to you in a wide variety of ways or with different keys or nested structures, it is important to include, as part of your prompt, exactly how you want the data to be returned. This is often most easily accomplished by providing a sample of how you’d like the content to be returned. You can embed this data structure into your prompt by just converting it to a string using `str()`.

Once you have retrieved the content from the NotOpenAI API response, your goal is to print to the screen the names of the oldest game and the youngest game included in the top 5. What specific games these end up corresponding to will vary some based on what the top 5 responses are each time you run the program!

If you are having issues, they are most often related to how the data is returned from the LLM. Make sure you have explained that you'd like JSON data back in your prompt, and given it the example of how it should be formatted. You can always print off just the `response_str` before converting it to a Python dictionary to check that it looks like what you'd expect.